Module Guide for Squigglr

Team #17, Squigglr
Rayan Nasrallah
Ahmed Elzaria
Muhammad Huzaifah
Shamil Canbolat
Moustafa Moustafa

November 13, 2026

1 Revision History

Date	Developer(s)	Change
9 November 2026	Moustafa	Added the reference material, the introduction and the anticipated and unlikely changes (#297).
9 November 2026	Moustafa	Added the module hierarchy, the connection between requirements and design and the module decomposition (#298).
9 November 2026	Moustafa	Added the traceability matrices and the uses hierarchy diagram (#299).
9 November 2026	Moustafa	Added the user interfaces, communication protocols, database design and timeline, and fixed the MG and writing checklist items (#300).
9 November 2026	Moustafa	Addressed the PR review: narrowed the Storage, Adult Access, Audio Output, Connection Recovery and Audit Log secrets, fixed four uses, redrew the uses hierarchy by level, and added missing modules to four trace rows (#302).
10 November 2026	Moustafa	Added who implements each module to the timeline, and set the year of the MG and MIS references to 2026 (#302).
10 November 2026	Moustafa	Addressed the Copilot review: listed every way a lesson ends in the Guided Lesson Control services, let Storage delete records, added the Attempt Record Module to SAF-RAY-004, and reworded AC25 and the introduction (#302).
11 November 2026	Ahmed Elzaria	Added FR-LRN-025 to FR-LRN-039, which the SRS split from FR-LRN-003 to 006, 009, 010, 019, 020 and 024, to the trace between requirements and modules, now 158 requirements (SRS #309).

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
BR	Business Rule (SRS)
DAG	Directed Acyclic Graph
FR	Functional Requirement (SRS)
HTTPS	Hypertext Transfer Protocol Secure
LC	Likely Change (SRS)
M	Module
MG	Module Guide
MIS	Module Interface Specification
NFR	Non-Functional Requirement (SRS)
OS	Operating System
PH	Delivery Phase (SRS)
SAF	Safety-Critical Requirement (SRS)
SRS	Software Requirements Specification
TBD	To Be Determined
UC	Use Case (SRS)
Squigglr	The drawing practice website this MG describes
ULC	Unlikely Change

Contents

1 Revision History **i**

2 Reference Material **ii**

2.1 Abbreviations and Acronyms . . . ii

3 Introduction **1**

4 Anticipated and Unlikely Changes **2**

4.1 Anticipated Changes . . . 2

4.2 Unlikely Changes . . . 3

5 Module Hierarchy **4**

6 Connection Between Requirements and Design **5**

7 Module Decomposition **8**

7.1 Hardware-Hiding Modules . . . 8

7.1.1 Input Device Module (M1) . . . 8

7.1.2 Display Module (M2) . . . 8

7.1.3 Audio Output Module (M3) . . . 9

7.2 Behaviour-Hiding Modules . . . 9

7.2.1 Guided Lesson Control Module (M4) . . . 9

7.2.2 Lesson Catalogue Module (M5) . . . 10

7.2.3 Attempt Module (M6) . . . 10

7.2.4 Attempt Evaluation Module (M7) . . . 10

7.2.5 Feedback Module (M8) . . . 10

7.2.6 Improvement Measure Module (M9) . . . 11

7.2.7 Attempt Record Module (M10) . . . 11

7.2.8 Creative Drawing Module (M11) . . . 11

7.2.9 Refinement Control Module (M12) . . . 11

7.2.10 Child Screen Module (M13) . . . 12

7.2.11 Adult Access Module (M14) . . . 12

7.2.12 Learner Profile Module (M15) . . . 12

7.2.13 Learner Settings Module (M16) . . . 12

7.2.14 Progress Summary Module (M17) . . . 13

7.2.15 Adult View Module (M18) . . . 13

7.3 Software Decision Modules . . . 13

7.3.1 Stroke Module (M19) . . . 13

7.3.2 Geometry Module (M20) . . . 14

7.3.3 Edit History Module (M21) . . . 14

7.3.4 Refinement Engine Module (M22) . . . 14

7.3.5 Storage Module (M23) . . . 14

7.3.6 Connection Recovery Module (M24) 15
7.3.7 Audit Log Module (M25) 15

8 Traceability Matrix **15**

9 Use Hierarchy Between Modules **18**

10 User Interfaces **20**

11 Design of Communication Protocols **20**

12 Database Design **21**

13 Timeline **21**

14 Generative AI Use Disclosure **22**
14.1 AI Tools Used 22
14.2 How and Where Used 23
14.3 Verification of AI-Generated Content 23

List of Tables

1 Module Hierarchy 6
2 Trace Between Requirements and Modules 15
3 Trace Between Anticipated Changes and Modules 18
4 Data kept by Squigglr 21
5 Timeline by module 22

List of Figures

1 Use hierarchy among modules. Shaded boxes are Hardware-Hiding modules, plain boxes are Behaviour-Hiding modules and dashed boxes are Software Decision modules. An arrow from A to B means A uses B. 19

3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (Parnas et al., 1984). We advocate a decomposition based on the principle of information hiding (Parnas, 1972). This principle supports design for change, because the "secrets" that each module hides represent likely future changes. Design for change matters for Squigglr because several parts are expected to change: the lessons, the supported devices, the refinement service and the feedback the child sees. These are listed in Section 27 of the Software Requirements Specification (SRS) (Squigglr Team, 2026b).

Our design follows the rules laid out by Parnas et al. (1984), as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the SRS, the Module Guide (MG) is developed (Parnas et al., 1984). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The requirements the modules meet are in the SRS (Squigglr Team, 2026b), and the interface of each module is in the Module Interface Specification (MIS) (Squigglr Team, 2026a). The potential readers of this document are as follows:

- New project members: This document can be a guide for a new project member to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers' understanding when they need to make changes to the system. A maintainer should update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the completeness

of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 9 describes the use relation between modules. Sections 10, 11, 12 and 13 cover the user interfaces, communication protocols, database design and timeline.

4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adopted here is called design for change.

The anticipated changes (AC) cover every likely change in the SRS (LC-001 to LC-008, SRS Section 27 (Squigglr Team, 2026b)), shown in brackets after each one, and add the design decisions the SRS does not fix. They are changes to the requirements, the design or the implementation, not to values set while Squigglr runs, such as a child's adult-set values.

AC1: The devices, browsers and input types Squigglr reads drawing input from, for example a new tablet or finger input added later (LC-003, LC-006).

AC2: How strokes, guides and images are drawn on the screen, for example a different drawing library.

AC3: How sounds are played in the browser.

AC4: The rules for moving through a guided lesson, for example when an extension is offered or when a go-back suggestion is made.

AC5: The lessons in the catalogue, their order, their target shapes and their predefined difficulties, for example adding letters after shapes (LC-001).

AC6: What ends an attempt, for example adding a button the child taps when done.

AC7: How an attempt is evaluated against its target and how difficulties are found, for example a new measure of how far a stroke is from the target.

AC8: The wording, images, colours and sounds of the feedback the child sees (LC-005).

AC9: How improvement is measured from past attempts (LC-008).

AC10: What an attempt or lesson record holds, for example a new value the adults want recorded.

AC11: The editing tools in the creative space, for example adding a fill tool.

AC12: When a drawing is refined and how a result is checked before it is applied, for example a new refinement tolerance check.

AC13: The layout and navigation of the child screens, which educators and therapists will review (NFR-UX-007).

AC14: How adults sign in and confirm adult-only actions, for example a school sign-in or a different authentication service.

AC15: What a learner profile holds and how the active learner is chosen, for example a guest learner.

AC16: Which settings a learner has, and the team-set range and default of each, which may be recalibrated after adult-made verification (LC-002).

AC17: Which recorded values a progress summary shows, how they are grouped, and whether a summary can be exported (LC-007, OI-02).

AC18: The layout and navigation of the adult views.

AC19: The data structure for strokes, for example adding stylus pressure.

AC20: The geometric algorithms on paths, for example a different smoothing method.

AC21: How edits are kept for undo and redo, for example how many steps are kept.

AC22: The algorithm or service that refines stroke coordinates, which may change because of cost, quality or availability (LC-004).

AC23: The database and hosting service.

AC24: Where unsaved attempts and drawings are kept on the device and when they are sent again.

AC25: How and where the audit log is kept.

4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the system architecture stage can simplify the software design. If these decisions should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

The unlikely changes cover every unlikely change in the SRS (ULC-001 to ULC-006, SRS Section 27 (Squigglr Team, 2026b)), shown in brackets after each one, and add two design decisions the modules rely on.

ULC1: Squigglr is a website used in a browser, with no installation (ULC-001).

ULC2: Only supporting adults view and manage practice records. Children do not (ULC-002).

ULC3: An adult chooses the lesson, the starting stage and any skip or backward move. Passing an attempt moves to the next stage of the same lesson, never to a new lesson (ULC-003).

ULC4: Squigglr gives no diagnosis, no judgement of a child's abilities, no comparison with other children and no effectiveness claims (ULC-004).

ULC5: Guided lesson records keep the strokes the child actually drew. Refinement happens only in the creative space, the original can be restored, and refinement can be turned off (ULC-005).

ULC6: The project gathers no information from or about children and runs no study with children (ULC-006).

ULC7: Each guided lesson has four guidance stages in a fixed order: demonstration, full outline, endpoints only and independent drawing (FR-LRN-002).

ULC8: Drawings are kept as paths of stroke coordinates, not as images, so they stay editable and can be evaluated and refined.

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets in Table 1. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented. Level 1 of the hierarchy (Hardware-Hiding, Behaviour-Hiding and Software Decision) only groups the leaves and is not implemented.

M1: Input Device Module

M2: Display Module

M3: Audio Output Module

M4: Guided Lesson Control Module

M5: Lesson Catalogue Module

M6: Attempt Module

M7: Attempt Evaluation Module

M8: Feedback Module

M9: Improvement Measure Module

M10: Attempt Record Module

M11: Creative Drawing Module

M12: Refinement Control Module

M13: Child Screen Module

M14: Adult Access Module

M15: Learner Profile Module

M16: Learner Settings Module

M17: Progress Summary Module

M18: Adult View Module

M19: Stroke Module

M20: Geometry Module

M21: Edit History Module

M22: Refinement Engine Module

M23: Storage Module

M24: Connection Recovery Module

M25: Audit Log Module

6 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the SRS. In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 2. The design decisions below are the ones the requirements call for but do not make themselves.

- **Lessons are data.** The lessons, stages, target shapes and predefined difficulties are kept as data that the Lesson Catalogue Module (M5) reads, so adding or reordering a lesson changes no code (NFR-MAINT-001, LC-001). All the rules for moving between stages are in one module, the Guided Lesson Control Module (M4), so a programmer can find stage behaviour in one place (NFR-MAINT-002).

Level 1	Level 2
Hardware-Hiding Module	Input Device Module
	Display Module
	Audio Output Module
Behaviour-Hiding Module	Guided Lesson Control Module
	Lesson Catalogue Module
	Attempt Module
	Attempt Evaluation Module
	Feedback Module
	Improvement Measure Module
	Attempt Record Module
	Creative Drawing Module
	Refinement Control Module
	Child Screen Module
	Adult Access Module
	Learner Profile Module
	Learner Settings Module
	Progress Summary Module
	Adult View Module
Software Decision Module	Stroke Module
	Geometry Module
	Edit History Module
	Refinement Engine Module
	Storage Module
	Connection Recovery Module
	Audit Log Module

Table 1: Module Hierarchy

- **One way in for drawing input.** Only the Input Device Module (M1) reads browser pointer events. The module drops duplicate reports of the same event (SAF-AHM-001), keeps points in place when the canvas size or orientation changes (SAF-AHM-002), and passes the rest of the system strokes in the form the Stroke Module (M19) defines. A new device or finger input then changes only M1 (LC-003, LC-006).
- **Strokes are drawn on the device.** The Display Module (M2) draws each stroke

as it comes in, with no call to the server, to meet the stroke feedback latency (NFR-PERF-001).

- **Feedback content is kept apart from feedback rules.** The Guided Lesson Control Module decides which feedback to give, and the Feedback Module (M8) holds what it looks and sounds like, so educators and therapists can revise the content without changing the rules (LC-005, NFR-UX-007).

- **Refinement is checked before it is applied.** The Refinement Control Module (M12) keeps the original strokes, checks every result from the Refinement Engine Module (M22) against the refinement tolerance, and leaves the drawing unchanged if the result is late, fails or is out of tolerance (FR-CRT-012, FR-CRT-017, FR-CRT-020, FR-CRT-021). The refinement service can then be swapped without touching the creative space (LC-004).

- **Records are written once.** The Attempt Record Module (M10) saves each attempt once and never changes a saved record (FR-REC-008, FR-REC-009). The Storage Module (M23) refuses changes to saved records other than deletion, which follows the privacy rules still to be decided (OI-02), and the Audit Log Module (M25) logs any attempt to change one (NFR-SEC-003).

- **A save counts only when the server confirms it.** The Storage Module reports a save as done only after the server confirms it (SAF-SHA-003). Until then, the Connection Recovery Module (M24) keeps the attempt or drawing on the device and sends it again when the connection comes back (SAF-RAY-003, NFR-PERF-005, FR-CRT-025).

- **Every request is from a signed-in adult.** The Adult Access Module (M14) signs adults in through an authentication service, and the Storage Module, given the signed-in adult by its caller, serves data only to the adult who owns it (SAF-SHA-001, NFR-SEC-001). An action a child must not do alone, such as choosing a lesson, goes through an adult check in M14 (SAF-RAY-001).

- **Learner settings are checked in one place.** The Learner Settings Module (M16) checks every value against its range and the other values before saving it (SAF-RAY-008), and the other modules only read the saved values (LC-002).

- **Progress summaries are not stored.** The Progress Summary Module (M17) builds a summary from the saved records each time an adult opens it, as the SRS data model describes, so a summary can never disagree with the records (G.6, LC-007).

- **Child screens and adult views are separate modules.** The Child Screen Module (M13) and the Adult View Module (M18) share no layout, which keeps the adult views distinct from what the child sees (NFR-UX-003).

7 Module Decomposition

Modules are decomposed according to the principle of "information hiding" proposed by Parnas et al. (1984). The *Secrets* field in a module decomposition is a brief statement of the design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it. For each module, a suggestion for the implementing software is given under the *Implemented By* title. If the entry is *OS*, this means that the module is provided by the operating system, the browser or standard programming language libraries. *Squigglr* means the module will be implemented by the Squigglr software, and a library in brackets is one the module is expected to use.

Where a secret lists several items, such as the wording, images, colours and sounds of feedback, they are parts of one design decision. Only the Input Device Module (M1) and the Guided Lesson Control Module (M4) hide two decisions, and their entries say why.

Only the leaf modules in the hierarchy have to be implemented. If a dash (–) is shown, this means that the module is not a leaf and will not have to be implemented.

7.1 Hardware-Hiding Modules

Secrets: The data structures and algorithms used to implement the virtual hardware.

Services: Serves as a virtual hardware used by the rest of the system. These modules provide the interface between the device, through the browser, and the software. So, the system can use them to accept drawing input, draw on the screen and play sounds.

Implemented By: –

7.1.1 Input Device Module (M1)

Secrets: The way the browser reports touchscreen and stylus input, and the list of supported device and browser combinations. These are one secret because a new device changes both together (LC-003).

Services: Turns browser pointer events into stroke points, drops duplicate reports of the same event, keeps points in place when the canvas size or orientation changes, and reports whether the device and browser are supported.

Implemented By: Squigglr (Pointer Events)

Type of Module: Abstract Object

7.1.2 Display Module (M2)

Secrets: The drawing library and the method used to draw strokes, guides and images on the screen.

Services: Draws strokes, guide outlines, endpoints, demonstrations and feedback images on the canvas, and limits flashing on child screens.

Implemented By: Squigglr (Paper.js)

Type of Module: Abstract Object

7.1.3 Audio Output Module (M3)

Secrets: How the browser plays sounds.

Services: Plays a sound, and mutes or unmutes sound.

Implemented By: Squigglr (browser audio)

Type of Module: Abstract Object

7.2 Behaviour-Hiding Modules

Secrets: The contents of the required behaviours.

Services: Includes programs that provide externally visible behaviour of the system as specified in the software requirements specification (SRS) documents. These modules serve as a communication layer between the hardware-hiding modules and the software decision modules. The programs in these modules will need to change if there are changes in the SRS.

Implemented By: –

7.2.1 Guided Lesson Control Module (M4)

Secrets: The rules for moving through a guided lesson: which stage comes next, when an extension or go-back suggestion is given, and when the lesson ends. These are one secret because the SRS rules for advancing and ending a lesson depend on each other (FR-LRN-019 to FR-LRN-021).

Services: Checks that the device and browser are supported before a lesson starts, starts a lesson at the stage an adult chooses, starts each attempt, decides the feedback after each attempt, moves to the next stage, offers an extension or a go-back suggestion, and ends the lesson at the attempt limit, when the independent stage is passed (FR-LRN-023), or after `LESSON_IDLE_END` with no input, following a check-in at `LESSON_IDLE_PROMPT` (SAF-RAY-009).

Implemented By: Squigglr

Type of Module: Abstract Object

7.2.2 Lesson Catalogue Module (M5)

Secrets: The lessons, their stages, target shapes and predefined difficulties, and the format they are stored in.

Services: Gives the list of approved lessons, and the target shape, guides and predefined difficulties of each stage.

Implemented By: Squigglr

Type of Module: Abstract Object

7.2.3 Attempt Module (M6)

Secrets: The events that end an attempt.

Services: Collects the strokes of one attempt, asks for an evaluation at each stylus lift, and ends the attempt when it passes, after `ATTEMPT_IDLE` with no input, or when it is interrupted, and gives the evaluation result of the attempt.

Implemented By: Squigglr

Type of Module: Abstract Data Type

7.2.4 Attempt Evaluation Module (M7)

Secrets: The method that classifies an attempt against its target.

Services: Decides whether an attempt is valid, complete and guided, whether it meets `STAGE_PASS`, and which predefined difficulties it shows.

Implemented By: Squigglr

Type of Module: Library

7.2.5 Feedback Module (M8)

Secrets: The wording, images, colours and sounds of the feedback the child sees.

Services: Gives the content to show or play for each kind of feedback, such as a passed stage, a missed part or effort on a non-passing attempt.

Implemented By: Squigglr

Type of Module: Library

7.2.6 Improvement Measure Module (M9)

Secrets: The measure of improvement over past attempts.

Services: Says whether the child is improving at a stage, from the results of their past attempts within `IMPROVE_WINDOW`.

Implemented By: Squigglr

Type of Module: Library

7.2.7 Attempt Record Module (M10)

Secrets: The contents of attempt and lesson records.

Services: Saves each attempt once, with its actual strokes, result, difficulties and the adult-set values in use, saves each lesson record and suggestion response, and reads back a learner's records.

Implemented By: Squigglr

Type of Module: Abstract Object

7.2.8 Creative Drawing Module (M11)

Secrets: The editing tools of the creative space.

Services: Draws strokes, selects, moves, resizes, recolours and erases a part, undoes and redoes a change, starts a new drawing, shows unsaved changes, and saves and reopens the active learner's drawings.

Implemented By: Squigglr

Type of Module: Abstract Object

7.2.9 Refinement Control Module (M12)

Secrets: The rules for when a drawing is refined and how a result is accepted.

Services: Refines a finished drawing when refinement is on, checks the result against the refinement tolerance, applies it as editable strokes, shows that the drawing is refined, keeps the original strokes through the Creative Drawing Module and restores them exactly, and leaves the drawing unchanged when refinement fails or the result is out of date.

Implemented By: Squigglr

Type of Module: Abstract Object

7.2.10 Child Screen Module (M13)

Secrets: The layout and navigation of the child screens.

Services: Shows the guided lesson screen and the creative space with large touch targets, images and icons instead of text, and the active learner's avatar.

Implemented By: Squigglr (React, Next.js)

Type of Module: Abstract Object

7.2.11 Adult Access Module (M14)

Secrets: How adults sign up, sign in and out, and confirm actions a child must not do alone.

Services: Signs an adult up, in and out, signs them out after `IDLE_TIMEOUT`, logs sign-in events, and runs the adult check before an action a child must not do alone.

Implemented By: Squigglr (Supabase Auth)

Type of Module: Abstract Object

7.2.12 Learner Profile Module (M15)

Secrets: The learner profile data, including which profile is active.

Services: Creates a learner profile with an identifier and an avatar, lists an adult's learners, and sets and gives the active learner.

Implemented By: Squigglr

Type of Module: Abstract Object

7.2.13 Learner Settings Module (M16)

Secrets: A learner's settings (the adult-set values and the audio and refinement switches), with the range and default of each and the rules between them.

Services: Saves a learner's settings only when each is in its range and the rules between them hold, fills in the default for a value the adult does not choose, and gives the saved values to the other modules. The refinement switch is the one setting the child can change (FR-CRT-030).

Implemented By: Squigglr

Type of Module: Abstract Object

7.2.14 Progress Summary Module (M17)

Secrets: The method that builds a progress summary from the records.

Services: Builds one learner's progress over time by lesson, keeps attempts with and without a guide apart, shows recurring difficulties, interrupted attempts and stages not attempted, opens the attempts behind it, and gives no overall score.

Implemented By: Squigglr

Type of Module: Library

7.2.15 Adult View Module (M18)

Secrets: The layout and navigation of the adult views.

Services: Shows the sign-in, profile, settings, lesson choice, progress and history views in plain wording, usable by keyboard and with resizable text, and tells the adult which attempts or drawings could not be saved.

Implemented By: Squigglr (React, Next.js)

Type of Module: Abstract Object

7.3 Software Decision Modules

Secrets: The design decision based on mathematical theorems, physical facts, or programming considerations. The secrets of these modules are *not* described in the SRS. The Connection Recovery Module (M24) and the Audit Log Module (M25) meet SRS requirements, but what they hide (the on-device store and how the log is stored) is not in the SRS.

Services: Includes data structure and algorithms used in the system that do not provide direct interaction with the user.

Implemented By: –

7.3.1 Stroke Module (M19)

Secrets: The data structure for strokes and their points.

Services: Creates a stroke, adds points with their times, and gives the points of a stroke back.

Implemented By: Squigglr

Type of Module: Abstract Data Type

7.3.2 Geometry Module (M20)

Secrets: The geometric algorithms on paths.

Services: Measures how far a path is from a target path and how much of the target it covers, and smooths and simplifies a path.

Implemented By: Squigglr (Paper.js)

Type of Module: Library

7.3.3 Edit History Module (M21)

Secrets: The data structure that keeps edits for undo and redo.

Services: Records a change, undoes the last change and redoes an undone change.

Implemented By: Squigglr

Type of Module: Abstract Data Type

7.3.4 Refinement Engine Module (M22)

Secrets: The algorithm or service that refines stroke coordinates.

Services: Takes stroke coordinates and returns adjusted coordinates at the same scale, or reports that it could not refine them.

Implemented By: Squigglr (refinement service TBD, LC-004)

Type of Module: Library

7.3.5 Storage Module (M23)

Secrets: The database and hosting service.

Services: Saves and reads the data each owner module gives it, for the signed-in adult its caller names, checks input before saving it, refuses changes to saved records other than deletion (FR-REC-009, OI-02), and confirms each save.

Implemented By: Squigglr (Supabase PostgreSQL)

Type of Module: Abstract Object

7.3.6 Connection Recovery Module (M24)

Secrets: How unsaved attempts and drawings are kept on the device until the server has them.

Services: Keeps an attempt or drawing on the device until the server confirms it is saved, sends it again when the connection comes back, and lists the ones that could not be saved.

Implemented By: Squigglr (browser storage)

Type of Module: Abstract Object

7.3.7 Audit Log Module (M25)

Secrets: How the audit log is stored.

Services: Logs sign-in events and attempts to change saved records on the server.

Implemented By: Squigglr

Type of Module: Abstract Object

8 Traceability Matrix

This section shows two traceability matrices: between the modules and the requirements and between the modules and the anticipated changes. Table 2 covers all 158 requirements in the SRS (Squigglr Team, 2026b): the functional requirements (FR), the non-functional requirements (NFR) and the safety-critical requirements (SAF). Every requirement is met by at least one module, and every module meets at least one requirement. NFR-MAINT-003 and NFR-MAINT-004 apply to every module ("All" in the table), because each module will have an MIS entry (#264 to #266) and follow the team's coding style.

Table 2: Trace Between Requirements and Modules

Req.	Modules	Req.	Modules
FR-LRN-001	M5	FR-PRG-001	M15, M17
FR-LRN-002	M4, M5	FR-PRG-002	M17
FR-LRN-003	M2, M4	FR-PRG-003	M17
FR-LRN-004	M2, M4	FR-PRG-004	M17
FR-LRN-005	M2, M4	FR-PRG-005	M17, M18
FR-LRN-006	M2, M4	FR-PRG-006	M17, M18
FR-LRN-007	M1, M6, M19	FR-PRG-007	M17, M18

Req.	Modules	Req.	Modules
FR-LRN-008	M6, M19	FR-PRG-008	M17
FR-LRN-009	M4, M6, M10	FR-PRG-009	M17
FR-LRN-010	M6, M7	FR-PRG-010	M17
FR-LRN-011	M7, M20	FR-PRG-011	M17
FR-LRN-012	M7	FR-PRG-012	M17
FR-LRN-013	M7, M20	NFR-UX-001	M13
FR-LRN-014	M4, M7, M8	NFR-UX-002	M13
FR-LRN-015	M4	NFR-UX-003	M13, M18
FR-LRN-016	M4, M14, M18	NFR-UX-004	M8
FR-LRN-017	M16, M18	NFR-UX-005	M13
FR-LRN-018	M4, M14	NFR-UX-006	M8, M13
FR-LRN-019	M4, M10	NFR-UX-007	M8, M13
FR-LRN-020	M4, M9	NFR-UX-008	M4, M13
FR-LRN-021	M4, M16	NFR-UX-009	M15, M18
FR-LRN-022	M8, M17	NFR-UX-010	M3, M16
FR-LRN-023	M4	NFR-UX-011	M12, M16
FR-LRN-024	M4, M6, M7	NFR-UX-012	M13, M18
FR-LRN-025	M4	NFR-UX-013	M2, M5
FR-LRN-026	M4, M6	NFR-UX-014	M18
FR-LRN-027	M2, M4	NFR-UX-015	M8
FR-LRN-028	M4, M6	NFR-UX-016	M8
FR-LRN-029	M4, M6	NFR-UX-017	M17, M18
FR-LRN-030	M4, M6	NFR-UX-018	M13
FR-LRN-031	M4	NFR-UX-019	M13, M18
FR-LRN-032	M6, M4	NFR-UX-020	M13, M18
FR-LRN-033	M4	NFR-UX-021	M2, M8
FR-LRN-034	M4, M10	NFR-UX-022	M3
FR-LRN-035	M4	NFR-UX-023	M18
FR-LRN-036	M4, M9	NFR-UX-024	M18
FR-LRN-037	M4	NFR-PERF-001	M1, M2
FR-LRN-038	M6, M7	NFR-PERF-002	M12, M22
FR-LRN-039	M6, M7	NFR-PERF-003	M17, M18, M23
FR-CRT-001	M1, M11, M19	NFR-PERF-004	M7, M20
FR-CRT-002	M11	NFR-PERF-010	M7
FR-CRT-003	M11	NFR-PERF-005	M23, M24
FR-CRT-004	M11	NFR-PERF-008	M23

Req.	Modules	Req.	Modules
FR-CRT-005	M11	NFR-PERF-009	M1, M2, M23
FR-CRT-006	M11	NFR-OPS-001	M14, M22, M23
FR-CRT-007	M11, M21	NFR-OPS-002	M14, M22, M23
FR-CRT-008	M11, M21	NFR-OPS-003	M13, M18
FR-CRT-009	M11	NFR-MAINT-001	M5
FR-CRT-010	M12, M22	NFR-MAINT-002	M4
FR-CRT-011	M12	NFR-MAINT-003	All
FR-CRT-012	M12, M20	NFR-MAINT-004	All
FR-CRT-013	M11, M12	NFR-SEC-001	M14, M23
FR-CRT-014	M12	NFR-SEC-002	M14
FR-CRT-015	M12, M22	NFR-SEC-003	M23, M25
FR-CRT-016	M12, M13	NFR-SEC-004	M15
FR-CRT-017	M12	NFR-SEC-005	M14, M25
FR-CRT-018	M12, M13	NFR-SEC-006	M23
FR-CRT-019	M12, M14, M18	NFR-CULT-002	M8, M13
FR-CRT-020	M12	NFR-COMP-001	M14, M15, M23
FR-CRT-021	M12	NFR-COMP-002	M15, M23
FR-CRT-022	M11, M12	NFR-COMP-005	M13, M18
FR-CRT-023	M11, M15, M23	SAF-RAY-001	M4, M14
FR-CRT-024	M11, M23	SAF-RAY-002	M1, M4, M18
FR-CRT-025	M11, M24	SAF-RAY-003	M18, M23, M24
FR-CRT-026	M11, M15	SAF-RAY-004	M10, M17
FR-CRT-027	M4, M12	SAF-RAY-005	M4, M16
FR-CRT-028	M10, M11	SAF-RAY-006	M4, M10, M17, M18
FR-CRT-029	M11, M13	SAF-RAY-007	M10, M13, M15
FR-CRT-030	M12, M16	SAF-RAY-008	M16
FR-REC-001	M10	SAF-RAY-009	M4, M6
FR-REC-002	M10, M19	SAF-AHM-001	M1
FR-REC-003	M10	SAF-AHM-002	M1
FR-REC-004	M10, M16	SAF-SHA-001	M14, M23
FR-REC-005	M6, M10	SAF-SHA-002	M14, M22, M23
FR-REC-006	M10	SAF-SHA-003	M18, M23, M24
FR-REC-007	M4, M10	SAF-SHA-006	M14
FR-REC-008	M10, M23, M24	SAF-MOU-001	M2, M8
FR-REC-009	M10, M23, M25	SAF-MOU-002	M10, M17
FR-REC-010	M4, M10	SAF-MOU-003	M2

Table 3 maps each anticipated change to the module that hides it. The mapping is one to one, so each anticipated change is hidden by exactly one module.

AC	Modules
AC1	M1
AC2	M2
AC3	M3
AC4	M4
AC5	M5
AC6	M6
AC7	M7
AC8	M8
AC9	M9
AC10	M10
AC11	M11
AC12	M12
AC13	M13
AC14	M14
AC15	M15
AC16	M16
AC17	M17
AC18	M18
AC19	M19
AC20	M20
AC21	M21
AC22	M22
AC23	M23
AC24	M24
AC25	M25

Table 3: Trace Between Anticipated Changes and Modules

9 Use Hierarchy Between Modules

In this section, the uses hierarchy between modules is provided. Parnas (1978) said of two programs A and B that A *uses* B if correct execution of B may be necessary for A to complete

the task described in its specification. That is, A *uses* B if there exist situations in which the correct functioning of A depends upon the availability of a correct implementation of B. Figure 1 illustrates the use relation between the modules. The graph is a directed acyclic graph (DAG). Each level of the hierarchy offers a testable and usable subset of the system, and modules in the higher level of the hierarchy are essentially simpler because they use modules from the lower levels.

An arrow from module A to module B means A uses B, so the code of A would import B. The figure is not a data flow diagram. Each band of the figure is one level of the hierarchy, labelled on the left. A module at level 0 uses no other module, and every other module is one level above the highest module it uses, so every arrow points down. Level 0 is drawn on two rows to leave room for the arrows. The Child Screen Module (M13) and the Adult View Module (M18) are at the top, because every action starts from a child screen or an adult view. The Guided Lesson Control Module (M4) is the control module for guided lessons and sits just below them. Modules that only take inputs and return results, such as the Attempt Evaluation Module (M7) and the Improvement Measure Module (M9), are given the values they need by the module that calls them instead of reading the learner's settings themselves, which keeps coupling low. The Refinement Control Module (M12) is told whether refinement is on in the same way.

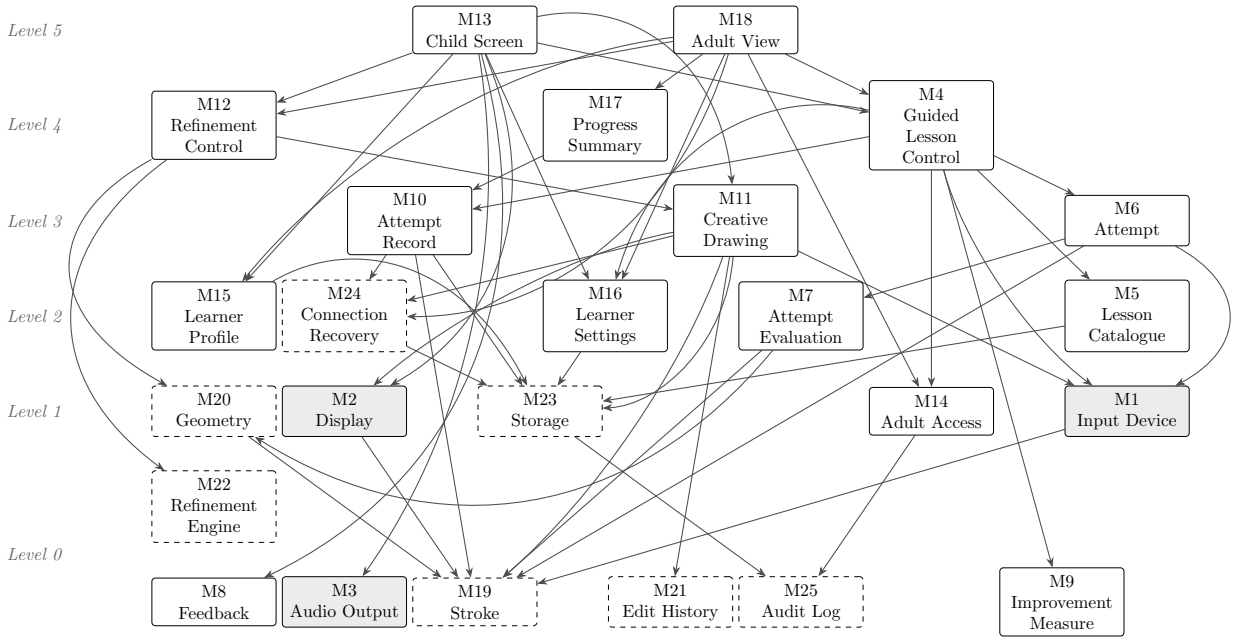


Figure 1: Use hierarchy among modules. Shaded boxes are Hardware-Hiding modules, plain boxes are Behaviour-Hiding modules and dashed boxes are Software Decision modules. An arrow from A to B means A uses B.

10 User Interfaces

Squigglr has two sets of screens, each owned by one module. The Child Screen Module (M13) owns the screens the child uses, and the Adult View Module (M18) owns the views the supporting adult uses. Each screen below names the SRS use cases it serves (Squigglr Team, 2026b).

- **Guided lesson screen (child):** the canvas with the demonstration, outline or endpoints for the current stage, the child's strokes and the feedback after each attempt, with the active learner's avatar always shown (UC-003, UC-008).
- **Creative space (child):** the canvas with the editing tools, undo and redo, refinement and the control to restore the original (UC-006, UC-007).
- **Sign-in and profiles (adult):** signing in, creating a child's profile and switching the active learner (UC-001).
- **Lesson choice (adult):** choosing the lesson and starting stage, and answering a go-back suggestion (UC-002, UC-004, UC-005).
- **Progress and history (adult):** the progress summary, the attempts behind it, each child's settings and the restore control for refined drawings (UC-009, UC-010).

The child screens use images and icons instead of text, large touch targets and a calm style (NFR-UX-001, NFR-UX-002, NFR-UX-018). The adult views use plain wording and work with a keyboard and resized text (NFR-UX-017, NFR-UX-023, NFR-UX-024). Screen layouts are not fixed yet, because the SRS asks for the style to be reviewed with educators or occupational therapists first (NFR-UX-007), so the designs will be added in Rev 0.

11 Design of Communication Protocols

There is no custom protocol. Drawing input reaches Squigglr through the browser's Pointer Events, which the Input Device Module (M1) reads. Everything else uses standard web protocols:

- The browser talks to the Squigglr server over HTTPS only (NFR-OPS-001).
- The server reaches the database and the authentication service through the hosting provider's client library over HTTPS, inside the Storage Module (M23) and the Adult Access Module (M14).
- The Refinement Engine Module (M22) sends stroke coordinates to the refinement service over HTTPS and gets adjusted coordinates back. No personal information is sent (SAF-SHA-002).

12 Database Design

Squigglr keeps its data in a hosted PostgreSQL database. The database follows the business data model in SRS Section 7 (Squigglr Team, 2026b), and each kind of data is read and written by one module only, through the Storage Module (M23). Table 4 summarizes it. The tables, keys and columns are left to the MIS and the implementation.

Data	Module	Notes
Adult accounts	M14	Kept by the authentication service.
Learner profiles	M15	An identifier and an avatar only (BR-001).
Learner settings	M16	The adult-set values of each learner.
Lessons	M5	Stages, target shapes and predefined difficulties.
Lesson and attempt records	M10	Added, never changed (FR-REC-009).
Creative drawings	M11	Strokes, plus the original strokes of a refined drawing.
Audit log	M25	Sign-in events and attempts to change saved records.

Table 4: Data kept by Squigglr

Access rules in the database let an adult read and write only their own learners' data (NFR-SEC-001). Progress summaries are not stored, as Section 6 explains.

13 Timeline

Table 5 gives, for each module, the team member who writes its MIS, the team member who implements it and the phases in which it is built. Each member implements five modules that work closely together, so the modules that use each other are mostly built by the same person. Ahmed builds the guided lessons, Shamil builds creative drawing and refinement, Muhammad builds the adult side, Moustafa builds strokes, records and storage, and Rayan builds the child screen, input and output. A module starts in the first phase that has one of its requirements and is finished by the last one, using the SRS phases: PH-1 is the proof of concept (November 25, 2026), PH-2 is Rev 0 (February 1, 2027) and PH-3 is the final demonstration (March 15, 2027) (Squigglr Team, 2026b). In PH-1, a module that a PH-1 module uses but that starts in PH-2 (M3, M9, M16, M24 and M25) is a stub that returns defaults. Implementation tasks are tracked as GitHub issues.

Module	MIS author	Implemented by	Starts	Done by
M1 Input Device	Shamil Canbolat	Rayan Nasrallah	PH-1	PH-3
M2 Display	Shamil Canbolat	Rayan Nasrallah	PH-1	PH-3
M3 Audio Output	Ahmed Elzaria	Rayan Nasrallah	PH-2	PH-2
M4 Guided Lesson Control	Ahmed Elzaria	Ahmed Elzaria	PH-1	PH-2
M5 Lesson Catalogue	Ahmed Elzaria	Ahmed Elzaria	PH-1	PH-2
M6 Attempt	Ahmed Elzaria	Ahmed Elzaria	PH-1	PH-2
M7 Attempt Evaluation	Ahmed Elzaria	Ahmed Elzaria	PH-1	PH-2
M8 Feedback	Ahmed Elzaria	Rayan Nasrallah	PH-1	PH-3
M9 Improvement Measure	Ahmed Elzaria	Ahmed Elzaria	PH-2	PH-2
M10 Attempt Record	Ahmed Elzaria	Moustafa Moustafa	PH-1	PH-2
M11 Creative Drawing	Shamil Canbolat	Shamil Canbolat	PH-1	PH-3
M12 Refinement Control	Shamil Canbolat	Shamil Canbolat	PH-1	PH-2
M13 Child Screen	Shamil Canbolat	Rayan Nasrallah	PH-1	PH-3
M14 Adult Access	Muhammad Huzaifah	Muhammad Huzaifah	PH-1	PH-3
M15 Learner Profile	Muhammad Huzaifah	Muhammad Huzaifah	PH-1	PH-3
M16 Learner Settings	Muhammad Huzaifah	Muhammad Huzaifah	PH-2	PH-2
M17 Progress Summary	Muhammad Huzaifah	Muhammad Huzaifah	PH-1	PH-3
M18 Adult View	Muhammad Huzaifah	Muhammad Huzaifah	PH-1	PH-3
M19 Stroke	Shamil Canbolat	Moustafa Moustafa	PH-1	PH-2
M20 Geometry	Ahmed Elzaria	Shamil Canbolat	PH-1	PH-2
M21 Edit History	Shamil Canbolat	Shamil Canbolat	PH-1	PH-3
M22 Refinement Engine	Shamil Canbolat	Shamil Canbolat	PH-1	PH-2
M23 Storage	Muhammad Huzaifah	Moustafa Moustafa	PH-1	PH-3
M24 Connection Recovery	Muhammad Huzaifah	Moustafa Moustafa	PH-2	PH-3
M25 Audit Log	Muhammad Huzaifah	Moustafa Moustafa	PH-2	PH-2

Table 5: Timeline by module

14 Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

14.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

14.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

14.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

References

David L. Parnas. On the criteria to be used in decomposing systems into modules. *Comm. ACM*, 15(2):1053–1058, December 1972.

David L. Parnas. Designing software for ease of extension and contraction. In *ICSE '78: Proceedings of the 3rd international conference on Software engineering*, pages 264–277, Piscataway, NJ, USA, 1978. IEEE Press. ISBN none.

D.L. Parnas, P.C. Clement, and D. M. Weiss. The modular structure of complex systems. In *International Conference on Software Engineering*, pages 408–419, 1984.

Squigglr Team. Module Interface Specification for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/Design/SoftDetailedDes/MIS.pdf, 2026a.

Squigglr Team. Software Requirements Specification for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/SRS/SRS.pdf, 2026b.